

Week 4: Make Data Model Ready

===== WEEK 4 START =====

Project: Airline Passenger Demand Forecasting using BTS T-100 Domestic Segment(All Carriers)

Week 4 objectives

- Process Week 3 training, validation, test partitions.
- Treat missing/invalid inputs and skewed numeric variables.
- Engineer past-only history and calendar features.
- Encode high-cardinality categorical predictors.
- Fit all learned preprocessing parameters on **training data only** and apply them unchanged to validation//test data.
- Make reproducibility/audit artifacts for the repository.

1. Environment and reusable Week 4 source

- Implementation stored in `Src/week4_src_model_ready.py`.
- Notebook calls that source directly so the submitted notebook / Python code execute same processing logic.

```
In [1]: from pathlib import Path
from IPython.display import display, Image
import importlib.util
import sys

def find_project_root(start=None):
    start = Path(start or Path.cwd()).resolve()
    for path in [start, *start.parents]:
        if (path / "Data").exists() and (path / "Src").exists() and (path / "Notebooks").exists():
            return path
    raise FileNotFoundError("Could not locate the project root.")

PROJECT_ROOT = find_project_root()
SRC_DIR = PROJECT_ROOT / "Src"
WEEK4_REPORTS_DIR = PROJECT_ROOT / "Reports" / "week4"
WEEK4_TABLES_DIR = WEEK4_REPORTS_DIR / "week4_tables"
WEEK4_FIGURES_DIR = WEEK4_REPORTS_DIR / "week4_figures"
WEEK4_PROCESSED_DIR = PROJECT_ROOT / "Data" / "processed" / "week4"

module_path = SRC_DIR / "week4_src_model_ready.py"
spec = importlib.util.spec_from_file_location("week4_model_ready", module_path)
week4 = importlib.util.module_from_spec(spec)
sys.modules["week4_model_ready"] = week4
spec.loader.exec_module(week4)

print("Project root:", PROJECT_ROOT)
print("Week 4 source:", module_path)
print("Week 4 reports:", WEEK4_REPORTS_DIR)
```

2. Rebuilds analytical table without rewriting prior-week reports

```
In [2]: monthly = week4.rebuild_monthly_without_rewriting_prior_reports(PROJECT_ROOT)
monthly = week4.add_past_only_features(monthly)

print(f"Carrier-route-month rows: {len(monthly):,}")
print(f"Reference period: {monthly['Period'].min():%Y-%m} through {monthly['Period'].max():%Y-%m}")
print("Past-only engineered columns present:")
print([c for c in ["PassengerLag1", "PassengerLag2", "PassengerRolling3Mean", "ScheduleCompletionRate"] if c in monthly.columns])

Carrier-route-month rows: 1,882,779
Reference period: 2015-01 through 2026-05
Past-only engineered columns present:
['PassengerLag1', 'PassengerLag2', 'PassengerRolling3Mean', 'ScheduleCompletionRate']
```

3. Recreate Week 3 chronological partitions

```
In [3]: (
    model_data,
    train_df,
    validation_df,
    test_df,
    split_summary,
) = week4.rebuild_week3_splits(monthly, PROJECT_ROOT)

display(split_summary)

print("Chronological integrity:")
```

```
print("Training < validation:", train_df[week4.TARGET_TIME_COLUMN].max() < validation_df[week4.TARGET_TIME_COLUMN].min())
print("Validation < test:", validation_df[week4.TARGET_TIME_COLUMN].max() < test_df[week4.TARGET_TIME_COLUMN].min())
```

	Split	Rows	ForecastMonths	FirstReferenceMonth	LastReferenceMonth	FirstForecastMonth	LastForecastMonth	RowPercent	MonthPercent
0	Training	1062603	95	2015-01-01	2022-11-01	2015-02-01	2022-12-01	67.842078	69.852941
1	Validation	239395	20	2022-12-01	2024-07-01	2023-01-01	2024-08-01	15.284216	14.705882
2	Test	264291	21	2024-08-01	2026-04-01	2024-09-01	2026-05-01	16.873706	15.441176

Chronological integrity:
Training < validation: True
Validation < test: True

4. Fit preprocessing on training data only. Apply unchanged to validation/test

Pipeline performs the following processing:

1. Treat invalid load factors and defensively flag invalid negative numeric values.
2. Fit numeric medians on training data only and impute missing model inputs.
3. Create exact-calendar passenger lags and a 3-month rolling mean with missing-history indicators.
4. Apply `log1p` to strongly skewed demand/capacity/count features.
5. Frequency-encode carrier, origin, and destination using training frequencies only.
6. Add cyclical forecast-month features and a deterministic time trend.
7. Standardize numeric model inputs using training-only mean/standard deviation.
8. Preserve both the raw target and `TargetLog1p` for later model comparison.

```
In [4]: results = week4.run_week4_from_splits(
    train_df,
    validation_df,
    test_df,
    split_summary,
    project_root=PROJECT_ROOT,
    save=True,
    figures_raw_train=train_df,
)

print("Training model-ready shape:", results.train_model_ready.shape)
print("Validation model-ready shape:", results.validation_model_ready.shape)
print("Test model-ready shape:", results.test_model_ready.shape)
print("Model feature count:", len(week4.model_feature_columns(results.train_model_ready)))
```

Training model-ready shape: (1062603, 32)
Validation model-ready shape: (239395, 32)
Test model-ready shape: (264291, 32)
Model feature count: 25

5. Processing summary

```
In [5]: display(results.processing_summary)
```

	Step	Processing	Implementation	LeakageControl
0	1	Preserve Week 3 chronological split	Same ordered forecast-month 70/15/15 split; no...	Future months remain isolated from training.
1	2	Treat invalid/missing numeric values	Out-of-range load factor and defensive negativ...	Validation/test values do not influence imputa...
2	3	Engineer exact-calendar history	Passenger lag-1, lag-2, and 3-month rolling me...	No future values and no silent bridging across...
3	4	Transform skewed numeric variables	log1p for demand/capacity/count features; raw ...	Transform is deterministic and target is never...
4	5	Encode high-cardinality categoricals	Carrier, origin, and destination are frequency...	Holdout category frequencies are never learned.
5	6	Encode seasonality and trend	Sine/cosine forecast-month features and a dete...	Uses calendar information known at forecast time.
6	7	Standardize numeric model inputs	StandardScaler parameters are fit on training ...	Future means/variances never enter training tr...

6. Model-ready feature manifest and sample

- Identifiers and date fields remain available for auditability.
- They'r excluded from numeric model-feature list.
- Model features themselves contain only numeric values.

```
In [6]: display(results.feature_manifest)

feature_cols = week4.model_feature_columns(results.train_model_ready)
print("Model features:")
print(feature_cols)

display(results.train_model_ready.head())
```

	Feature	Family	Treatment
0	z_PassengersLog1p	Standardized numeric	Training-median imputation where needed; optio...
1	z_SeatsLog1p	Standardized numeric	Training-median imputation where needed; optio...
2	z_DepScheduledLog1p	Standardized numeric	Training-median imputation where needed; optio...
3	z_DepPerformedLog1p	Standardized numeric	Training-median imputation where needed; optio...
4	z_PassengerLag1Log1p	Standardized numeric	Training-median imputation where needed; optio...
5	z_PassengerLag2Log1p	Standardized numeric	Training-median imputation where needed; optio...
6	z_PassengerRolling3MeanLog1p	Standardized numeric	Training-median imputation where needed; optio...
7	z_Distance	Standardized numeric	Training-median imputation where needed; optio...
8	z_LoadFactor	Standardized numeric	Training-median imputation where needed; optio...
9	z_ScheduleCompletionRate	Standardized numeric	Training-median imputation where needed; optio...
10	z_TrendMonth	Standardized numeric	Training-median imputation where needed; optio...
11	TargetMonthSin	Calendar seasonality	Deterministic cyclical encoding of known forec...
12	TargetMonthCos	Calendar seasonality	Deterministic cyclical encoding of known forec...
13	UniqueCarrierFreq	Categorical encoding	Training-only frequency encoding; unseen holdo...
14	OriginFreq	Categorical encoding	Training-only frequency encoding; unseen holdo...
15	DestFreq	Categorical encoding	Training-only frequency encoding; unseen holdo...
16	LoadFactorMissing	Missingness/data-quality indicator	Binary indicator preserves information about m...
17	LoadFactorInvalid	Missingness/data-quality indicator	Binary indicator preserves information about m...
18	PassengerLag1Missing	Missingness/data-quality indicator	Binary indicator preserves information about m...
19	PassengerLag2Missing	Missingness/data-quality indicator	Binary indicator preserves information about m...
20	PassengerRolling3MeanMissing	Missingness/data-quality indicator	Binary indicator preserves information about m...
21	ScheduleCompletionRateMissing	Missingness/data-quality indicator	Binary indicator preserves information about m...
22	UniqueCarrierMissing	Missingness/data-quality indicator	Binary indicator preserves information about m...
23	OriginMissing	Missingness/data-quality indicator	Binary indicator preserves information about m...
24	DestMissing	Missingness/data-quality indicator	Binary indicator preserves information about m...

Model features:

```
['z_PassengersLog1p', 'z_SeatsLog1p', 'z_DepScheduledLog1p', 'z_DepPerformedLog1p', 'z_PassengerLag1Log1p', 'z_PassengerLag2Log1p', 'z_PassengerRolling3MeanLog1p', 'z_Distance', 'z_LoadFactor', 'z_ScheduleCompletionRate', 'z_TrendMonth', 'TargetMonthSin', 'TargetMonthCos', 'UniqueCarrierFreq', 'OriginFreq', 'DestFreq', 'LoadFactorMissing', 'LoadFactorInvalid', 'PassengerLag1Missing', 'PassengerLag2Missing', 'PassengerRolling3MeanMissing', 'ScheduleCompletionRateMissing', 'UniqueCarrierMissing', 'OriginMissing', 'DestMissing']
```

	Period	TargetPeriod	UniqueCarrier	Origin	Dest	TargetPassengersNextMonth	TargetLog1p	z_PassengersLog1p	z_SeatsLog1p	z_DepScheduledLog1p	...	DestFreq
0	2015-01-01	2015-02-01	0JQ	GCN	LAS	409.0	6.016157	-1.831029	-1.568190	-1.799323	...	0.018625
1	2015-01-01	2015-02-01	0JQ	LAS	GCN	386.0	5.958425	-1.831029	-1.568190	-1.799323	...	0.000016
2	2015-01-01	2015-02-01	1DQ	ACK	HYA	2530.0	7.836370	0.209097	0.151700	1.761334	...	0.000342
3	2015-01-01	2015-02-01	1DQ	HYA	ACK	2392.0	7.780303	0.197085	0.174205	1.753698	...	0.000925
4	2015-01-01	2015-02-01	1PQ	HNL	LUP	161.0	5.087596	-1.023859	-0.864683	0.446518	...	0.000245

5 rows × 32 columns



7. Missingness and invalid-value treatment audit

```
In [7]: display(results.missingness_before_after)
```

	Split	Variable	MissingBefore	MissingAfter
0	Training	LoadFactor	1413	0
1	Training	PassengerLag1	90553	0
2	Training	PassengerLag2	135160	0
3	Training	PassengerRolling3Mean	156806	0
4	Training	ScheduleCompletionRate	91063	0
5	Validation	LoadFactor	270	0
6	Validation	PassengerLag1	17010	0
7	Validation	PassengerLag2	25059	0
8	Validation	PassengerRolling3Mean	29477	0
9	Validation	ScheduleCompletionRate	7448	0
10	Test	LoadFactor	186	0
11	Test	PassengerLag1	18183	0
12	Test	PassengerLag2	25682	0
13	Test	PassengerRolling3Mean	30715	0
14	Test	ScheduleCompletionRate	7801	0

8. Holdout-category audit

- Frequency maps learned from training only.
- Any category that first appears in validation or test receives frequency `0.0`. This stops causing error or being learned from future data.

In [8]: `display(results.unknown_category_audit)`

	Split	Variable	Rows	UnknownCategoryRows	UnknownCategoryPercent
0	Training	UniqueCarrier	1062603	0	0.000000
1	Training	Origin	1062603	0	0.000000
2	Training	Dest	1062603	0	0.000000
3	Validation	UniqueCarrier	239395	119	0.049709
4	Validation	Origin	239395	79	0.033000
5	Validation	Dest	239395	90	0.037595
6	Test	UniqueCarrier	264291	146	0.055242
7	Test	Origin	264291	347	0.131295
8	Test	Dest	264291	350	0.132430

9. Training-fitted preprocessing parameters

- Documents medians / scaling coefficients learned from training partition.
- Validation and test data are transformed using these frozen values.

In [9]: `display(results.preprocessor_parameters)`

	ParameterType	Variable	Value
0	Training median	Passengers	2507.0
1	Training median	Seats	3420.0
2	Training median	DepScheduled	31.0
3	Training median	DepPerformed	32.0
4	Training median	Distance	594.0
5	Training median	LoadFactor	0.792
6	Training median	PassengerLag1	2784.0
7	Training median	PassengerLag2	2859.0
8	Training median	PassengerRolling3Mean	3031.666667
9	Training median	ScheduleCompletionRate	1.0
10	Scaler mean	PassengersLog1p	7.311347
11	Scaler scale	PassengersLog1p	2.154009
12	Scaler mean	SeatsLog1p	7.747115
13	Scaler scale	SeatsLog1p	1.8028
14	Scaler mean	DepScheduledLog1p	3.31873
15	Scaler scale	DepScheduledLog1p	1.459206
16	Scaler mean	DepPerformedLog1p	3.496902
17	Scaler scale	DepPerformedLog1p	1.154248
18	Scaler mean	PassengerLag1Log1p	7.491332
19	Scaler scale	PassengerLag1Log1p	1.983989
20	Scaler mean	PassengerLag2Log1p	7.518546
21	Scaler scale	PassengerLag2Log1p	1.961339
22	Scaler mean	PassengerRolling3MeanLog1p	7.66923
23	Scaler scale	PassengerRolling3MeanLog1p	1.796657
24	Scaler mean	Distance	733.07955
25	Scaler scale	Distance	615.14539
26	Scaler mean	LoadFactor	0.712527
27	Scaler scale	LoadFactor	0.227775
28	Scaler mean	ScheduleCompletionRate	0.98762
29	Scaler scale	ScheduleCompletionRate	0.469077
30	Scaler mean	TrendMonth	46.33397
31	Scaler scale	TrendMonth	27.455437
32	Trend origin	TargetPeriod	2015-02-01

10. Data-quality checks on all splits

```
In [ ]: display(results.data_quality_checks)
assert results.data_quality_checks["Passed"].all(), "At least 1 Week4 quality check failed."
```

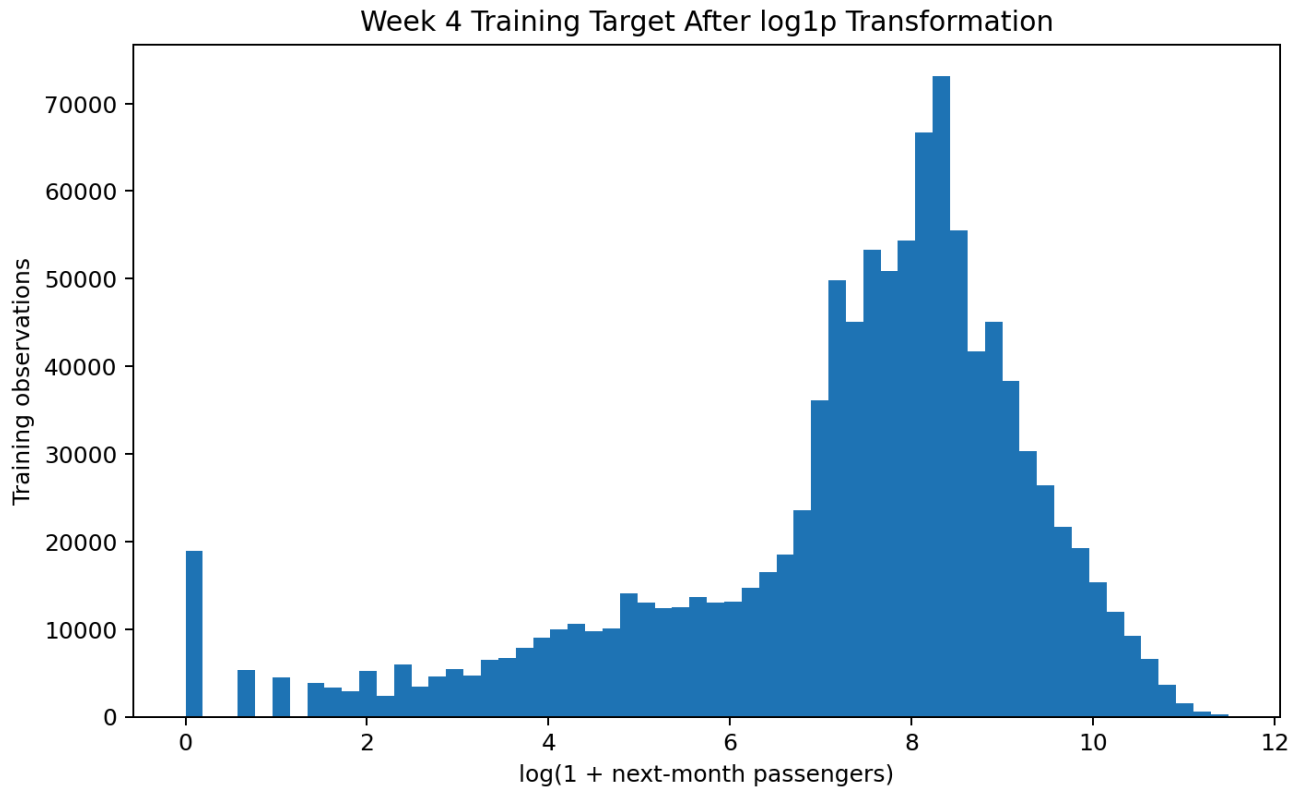
	Split	Check	Passed	Value
0	Training	Row count retained	True	1062603
1	Training	No missing model features	True	0
2	Training	No infinite model features	True	0
3	Training	Target remains observed	True	0
4	Validation	Row count retained	True	239395
5	Validation	No missing model features	True	0
6	Validation	No infinite model features	True	0
7	Validation	Target remains observed	True	0
8	Test	Row count retained	True	264291
9	Test	No missing model features	True	0
10	Test	No infinite model features	True	0
11	Test	Target remains observed	True	0

11. Week 4 figures

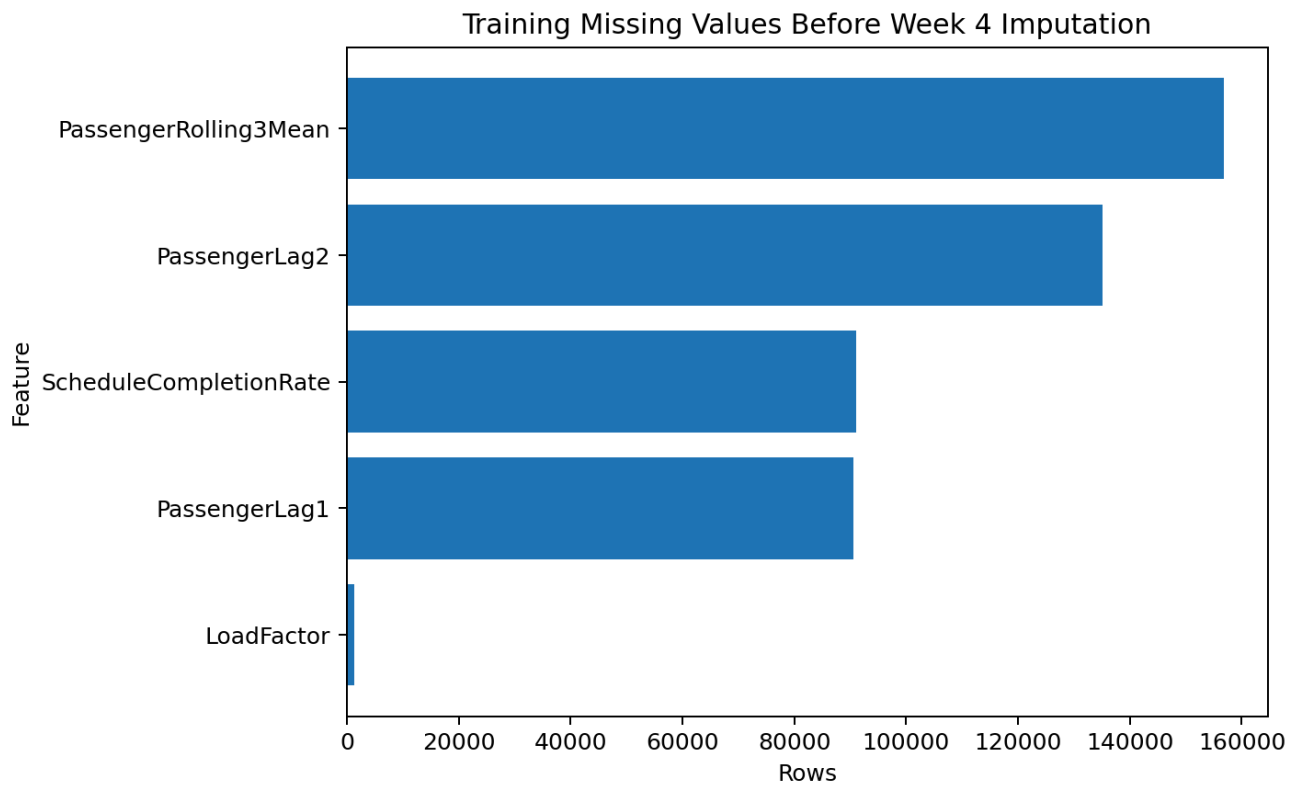
```
In [11]: figure_paths = [
    WEEK4_FIGURES_DIR / "week4_figure_training_target_log1p.png",
    WEEK4_FIGURES_DIR / "week4_figure_training_missing_before_imputation.png",
    WEEK4_FIGURES_DIR / "week4_figure_split_row_retention.png",
    WEEK4_FIGURES_DIR / "week4_figure_holdout_unknown_categories.png",
]

for path in figure_paths:
    if path.exists():
        print(path.name)
        display(Image(filename=str(path)))
```

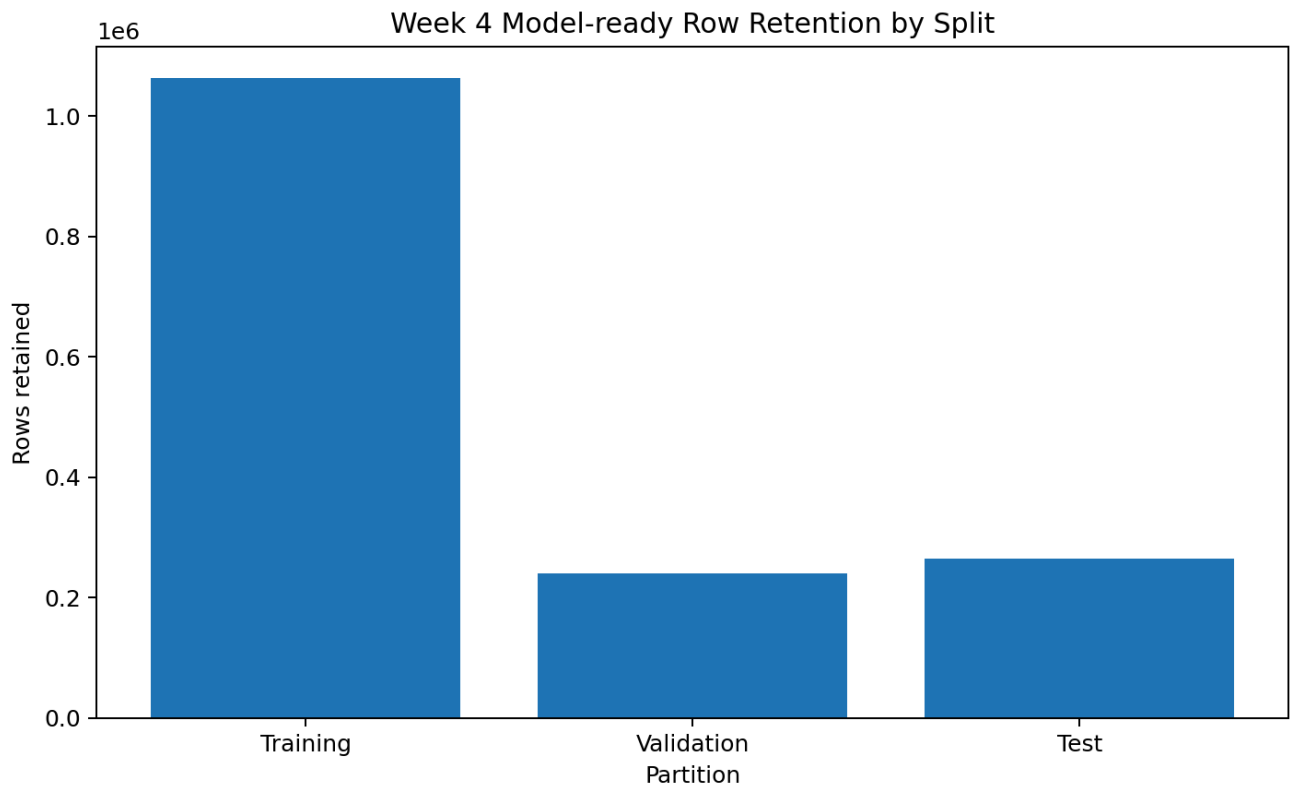
week4_figure_training_target_log1p.png



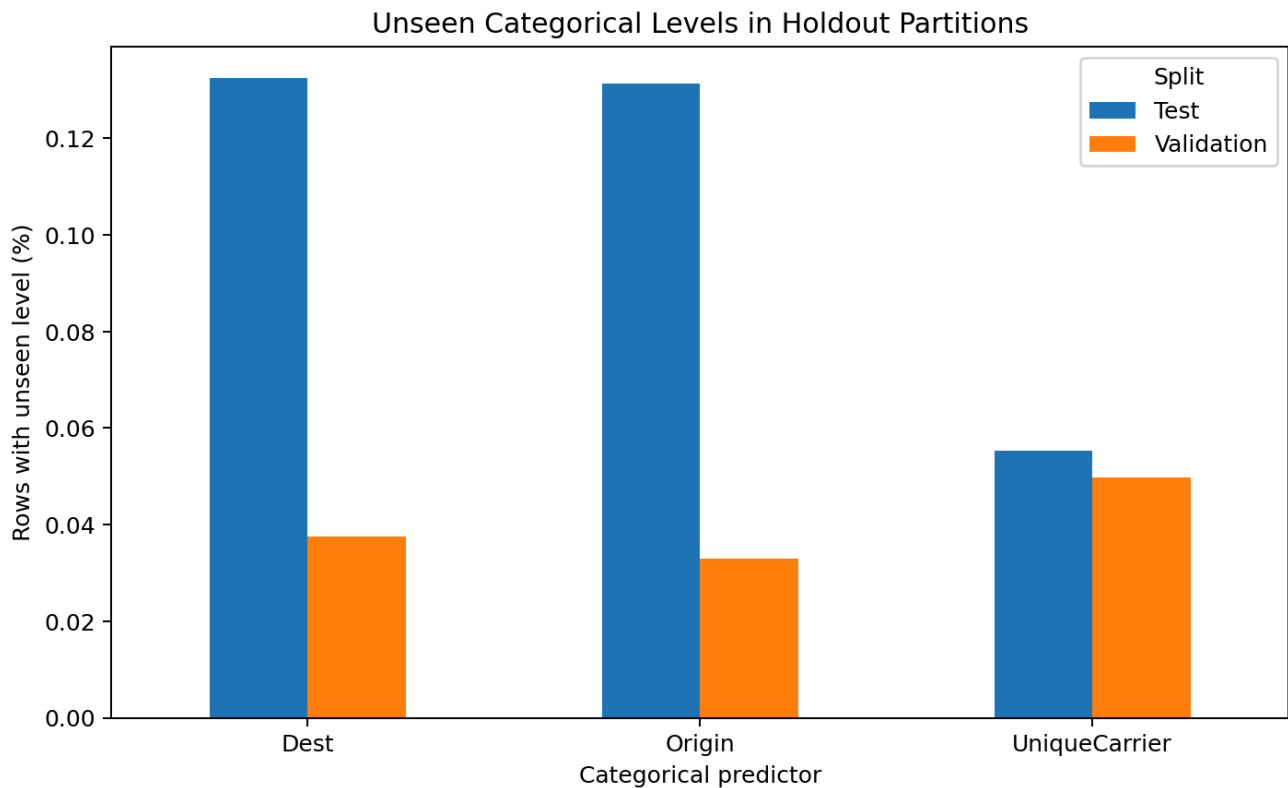
week4_figure_training_missing_before_imputation.png



week4_figure_split_row_retention.png



week4_figure_holdout_unknown_categories.png



12. Week 4 artifacts

Only representative processed-data samples pushed to GitHub.

```
In [12]: print("Week 4 tables:")
for path in sorted(WEEK4_TABLES_DIR.glob("*")):
    print("-", path.name)

print("\nWeek 4 figures:")
for path in sorted(WEEK4_FIGURES_DIR.glob("*.png")):
    print("-", path.name)

print("\nProcessed-data samples:")
for path in sorted(WEEK4_PROCESSED_DIR.glob("*")):
    print("-", path.name)
```

Week 4 tables:

- week4_category_frequency_maps.csv
- week4_data_quality_checks.csv
- week4_feature_manifest.csv
- week4_missingness_audit.csv
- week4_preprocessing_metadata.json
- week4_preprocessor_parameters.csv
- week4_processing_summary.csv
- week4_split_summary.csv
- week4_unknown_category_audit.csv

Week 4 figures:

- week4_figure_holdout_unknown_categories.png
- week4_figure_split_row_retention.png
- week4_figure_training_missing_before_imputation.png
- week4_figure_training_target_log1p.png

Processed-data samples:

- README.md
- week4_test_model_ready_sample.csv
- week4_training_model_ready_sample.csv
- week4_validation_model_ready_sample.csv

13. Week 4 completion checks

```
In [13]: assert len(results.train_model_ready) == int(split_summary.loc[split_summary["Split"].eq("Training"), "Rows"].iloc[0])
assert len(results.validation_model_ready) == int(split_summary.loc[split_summary["Split"].eq("Validation"), "Rows"].iloc[0])
assert len(results.test_model_ready) == int(split_summary.loc[split_summary["Split"].eq("Test"), "Rows"].iloc[0])

for frame in [results.train_model_ready, results.validation_model_ready, results.test_model_ready]:
    features = week4.model_feature_columns(frame)
    assert not frame[features].isna().any().any()

assert (WEEK4_TABLES_DIR / "week4_preprocessing_metadata.json").exists()
```

```
assert (WEEK4_FIGURES_DIR / "week4_figure_training_target_log1p.png").exists()
assert (WEEK4_PROCESSED_DIR / "week4_training_model_ready_sample.csv").exists()

print("All Week 4 checks passed.")
print("==== WEEK 4 END ====")
```

All Week 4 checks passed.
==== WEEK 4 END =====